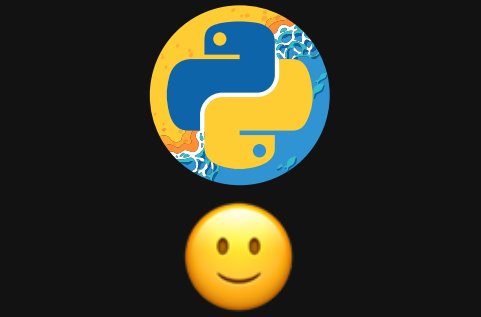
One smiley, a lot of RAM

The cost of making a string friendlier

Bernát Gábor

PyBeach 2026



Slides

Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.

“Just add one smiley”

```
In [2]: from sys import getsizeof

In [3]: text = "a" * 1_000_000

In [4]: getsizeof(text)
Out[4]: 1000041

In [5]: friendly = text + "😀 "

In [6]: getsizeof(friendly)
Out[6]: 4000064
```

`sys.getsizeof` counts this object's bytes.

Recorded CPython 3.15.0 / Linux x86-64

Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.

Friendliness +1. Memory ×4.

Same million "a" characters. One added smiley.



One smiley. About 3 MB of hospitality.

Measured: 1,000,041 and 4,000,064 bytes. Decimal MB. Includes object overhead. sys.getsizeof

Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.

From 1 Byte to 4 Bytes — Storage Measured in Text

Same million "a" characters. One added smiley.

a

U+0061



1 byte / code point

Before á

U+00E1



1.00 MB

1 byte / code point

After 🙂 €

U+20AC



4.00 MB

2 bytes / code point

One smiley. About 3 MB of hospitality.

🙂

U+1F642



4 bytes / code point

Each row is a separate string. One width **for every code point in it**.

Measured in UTF-8 bytes. Python 3.14 flexible string representation. PEP 393. 1,000,000 ASCII characters = 1,000,000 bytes. U+0000–U+00FF = 1 byte. U+0100–U+FFFF = 2 bytes. U+10000–U+10FFFF = 4 bytes. One non-BMP character upgrades the entire string.

Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.

Unicode text comes first

`text + "🙂"`

Unicode str + str



`friendly`

Unicode str



Each small compartment is one byte. `...` abbreviates repeated `"a"` characters.

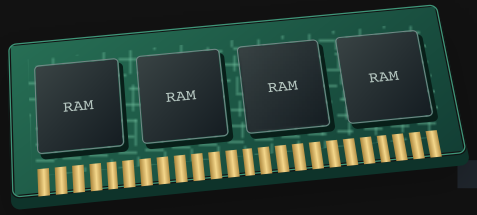
```
In [4]: friendly.isascii()
Out[4]: False
```

Python 3 str contains Unicode text. ASCII covers 128 code points; adding a smiley goes beyond them.

Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.

Same text. How many bytes?

Python `str`



`friendly`

a a … a 🙂

4.00 MB

Python object

The original stays in memory.

`.encode("utf-8")`



UTF-8 examples (hex)

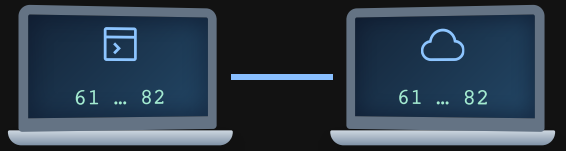
"a" → `61`

🙂 → `F0 9F 99 82`



File **? bytes**

UTF-8 payload



Network **? bytes**

UTF-8 payload

Text encodings / File bytes / Socket bytes. File/network overhead excluded. Hex bytes.

Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.

Same text, encoded as UTF-8. How many bytes?

```
In [4]: getsizeof(friendly)
Out[4]: 4000064

In [5]: encoded = friendly.encode("utf-8")

In [6]: len(encoded)
Out[6]: 1000004
```

`encode("utf-8")` creates a separate `bytes` object.

Same million "a" characters and smiley.

Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.

Economy in transit. Business in RAM.

`friendly` = 1,000,000 × `"a"` + 🙂

CPython str

In memory



4,000,004
payload bytes

UTF-8 bytes

After encoding



1,000,004
payload bytes

UTF-8 is smaller here: mostly-ASCII text.

Other text can be smaller in CPython's fixed-width slots.

```
In [5]: encoded.decode("utf-8") == friendly
Out[5]: True
```

CPython storage / ASCII and UTF-8. Payload bytes only; object headers and terminators excluded.

Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.

Refund approved

```
In [4]: getsizeof(friendly)
Out[4]: 4000064

In [5]: plain = friendly[:-1]

In [6]: getsizeof(plain)
Out[6]: 1000041

In [7]: getsizeof(friendly)
Out[7]: 4000064
```

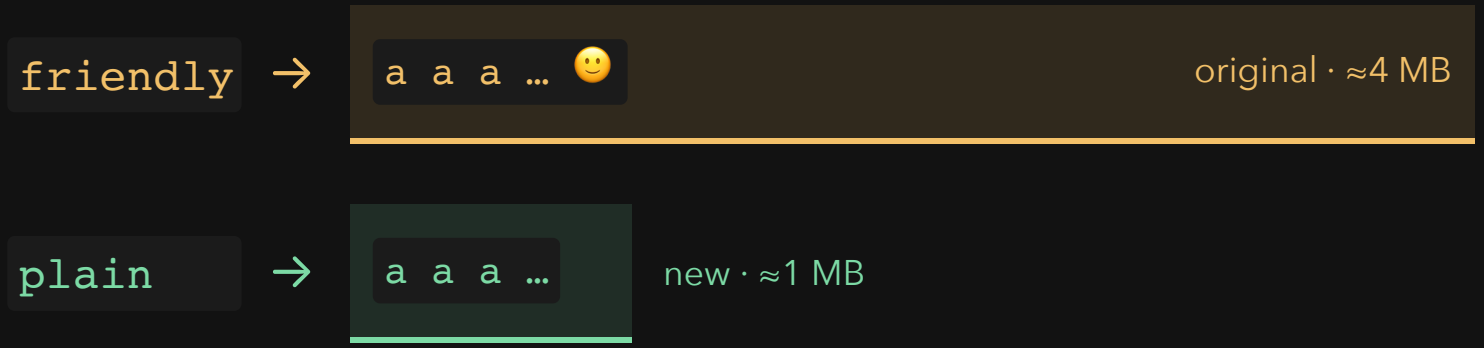
Slicing creates a **new string** that can use narrower storage.

The original string is unchanged.

Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.

A smaller result. Two live objects.

```
plain = friendly[:-1]
```



These two objects: **≈5.00 MB together**

while both names reference them.



Object lifetimes. These two string objects only; other objects and process memory excluded. Decimal MB.

Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.

The smiley can stay

```
friendly = "a" * 1_000_000 + "🙂"
```

`len(friendly)`	Code points	1,000,001
`len(encoded)`	UTF-8 payload bytes	1,000,004
`sys.getsizeof(friendly)`	String object bytes	4,000,064

Three questions. Three perfectly reasonable bills.

str / Unicode HOWTO / sys.getsizeof

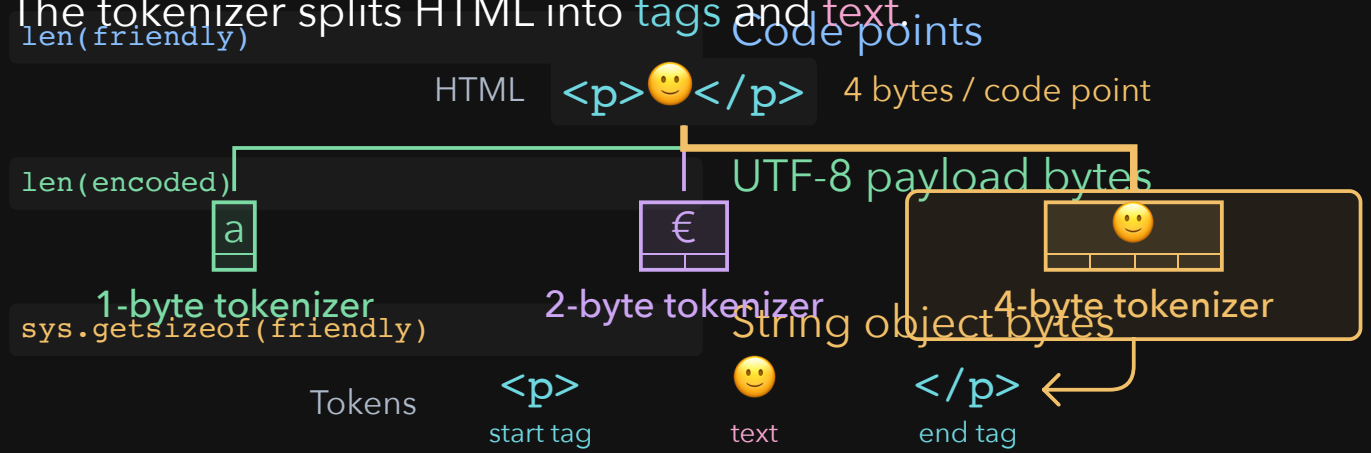
Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.

turbohtml

Fast HTML tools for Python 🙂. Typed API, C core.

Parse · query · clean · rewrite HTML

The tokenizer splits HTML into tags and text.



Three questions. Three perfectly reasonable bills.

Choose once. Skip the width check for each character.

https://bernat.tech/posts/blazing-fast-html-parser/

Keep the smiley. Measure the bill.

str / Unicode HOWTO / sys.getsizeof

Slides

Bloomberg
©2026 Bloomberg Finance L.P.
All rights reserved.